

Credit Risk Segmentation

Predicting Probability of Default from a Statistical, Machine Learning, and Deep Learning Perspective

Julio Cesar Carrillo Soto
julioccs@udel.edu

Abstract—This project addresses loan risk modeling by estimating the probability that a current borrower will default. This analysis will use the borrower and loan level information to build and evaluate predictive models from statistical, machine learning, and deep learning methods. Methods include logistic regression, random forest classifier, XGBoost classifier, and neural networks. The goal is not only to compare each model’s predictive performance, but also to interpret the key factors influencing default risk and develop a practical scorecard for stakeholder use.

I. INTRODUCTION

Credit risk modeling is a crucial task in the financial industry because it helps lenders estimate the probability that a borrower will default and identify which factors contribute most to that risk. Building an effective model allows financial institutions, such as banks, to manage risk in a more informed way. More specifically, these models can help analyze current customer behavior, guide policies for future customers, estimate expected loss, and allocate resources more effectively.

This project focuses on predicting whether current loan customers are likely to default using borrower and loan information. The response variable is derived from loan status, where each status is mapped into either a good-standing or default-risk category. We thus reduce the multiclass problem into a simple binary classification problem.

Several modeling approaches were used to address this classification problem from both a statistical and data science perspective. Specifically, this project will compare logistic regression, XGBoost, random forest, and neural networks of varying sizes based on their ability to classify default-risk loans. These models were evaluated using classification metrics that are appropriate for imbalanced data, such as the F1-score, ROC-AUC, precision, and recall.

In addition to comparing model performance, this project will develop an interpretable scorecard for stakeholder use. The scorecard will translate model results into a practical and easy-to-understand tool that can support decision-making.

II. METHODS

A. Problem setup

For this problem, the dependent variable is loan status, which originally contains seven categories, such as Current, Fully Paid, and Late. To make the problem more manageable and practical, these categories were mapped into two groups representing good-standing and default-risk loans. The dataset has shape (2029952, 142), giving 142 potential predictor variables. These predictors include both borrower and loan

information, such as loan amount, installment, grade, income, and interest rate.

Cleaning the data was challenging because the preprocessing needed to be performed inside the pipeline to reduce data leakage and ensure that any transformations were learned only from the training data. Highly associated categorical variables were identified using Cramér’s V, and mutual information was used to decide which variable to drop with respect to the dependent variable. Similarly, for continuous variables, Pearson correlation was used to identify highly linearly associated features, and the variable with the lower mutual information with respect to the dependent variable was dropped. After constructing the preprocessing and feature engineering pipeline, leakage variables were removed because they could reveal information about the loan outcome that would not be available at prediction time, giving the model optimistic and unrealistic results. Numerical variables were scaled using a RobustScaler, which is appropriate because many variables may contain outliers, and checking normality for more than 100 variables is not practical and computationally expensive. Categorical variables were one-hot encoded with the first category dropped. For numerical imputation, an indicator function was added to preserve information about whether a value was originally missing or imputed. Similarly, an indicator function was used to flag outlier points in numerical values using Tukey’s quartiles-based outlier detection.

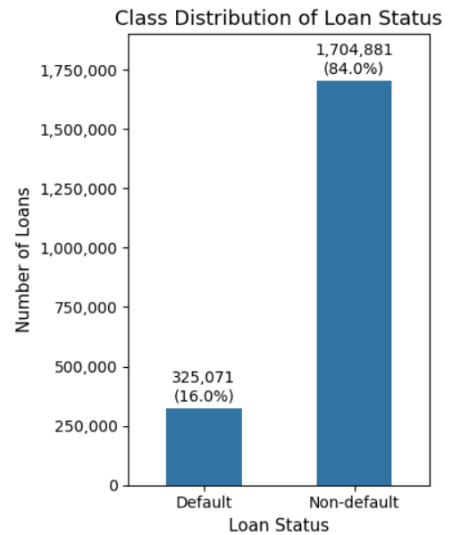


Fig. 1. Dependent variable class dist

B. Models Used

This project compares models from three different approaches: statistics, machine learning, and deep learning. Logistic regression was used as the main statistical model. Logistic regression is a statistical classification model commonly used when the response variable is categorical, especially binary. In this project, logistic regression estimates the probability that a loan belongs to the default risk class, based on loan and borrower information. This model was included because of its interpretability, since it provides a clear parameterized formula and coefficients that help explain how different variables affect default risk. It is also beneficial for the creation of the scorecard because the coefficients can be translated into a practical risk-scoring system for industry use.

Logistic regression estimates the probability of default using the following form:

$$P(Y = 0 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p)}} \quad (1)$$

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p \quad (2)$$

In equation 1, $P(Y = 0 | X)$ represents the probability that a loan belongs to the default-risk class given the predictor variables. The variables $X_1, X_2, \dots, X_p$ represent independent variables, while $\beta_0, \beta_1, \dots, \beta_p$ are parameters estimated from the data.

A decision tree is a tree-based machine learning algorithm that makes predictions by asking a series of questions about the input variables. Based on the answers, an observation moves through the branches of the tree until it reaches a final classification. The tree is built by repeatedly selecting the feature and split that best separates the classes, creating decision paths that help classify new observations.

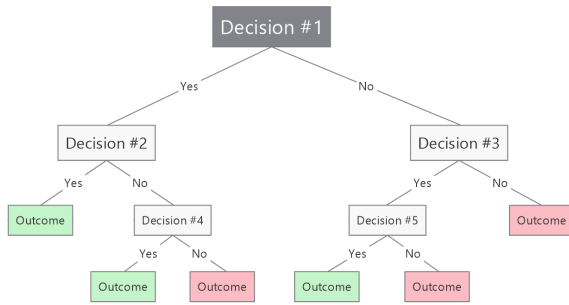


Fig. 2. Decision tree figure

Random forest is an ensemble method that builds many decision trees using bootstrap samples of the training data. At each split, only a random subset of features is considered to help reduce correlation between trees. This reduces correlation among trees, making averaging their predictions more effective, which lowers variance and can improve generalization.

The final prediction is made by aggregating the predictions across all built trees. Out-of-bag observations are used as an internal validation estimate by evaluating each tree on observations that were not included in the bootstrap sample.

XGBoost builds an ensemble of decision trees sequentially. It starts with an initial prediction and then evaluates the prediction error via the loss function. Gradient information from the loss function is then used to build new trees that correct the errors made by the previous model. Each new tree is added to the ensemble with a learning rate, while regularization is used to penalize overly complex trees and reduce overfitting.

C. Modeling and evaluation

As stated in the introduction, several models were used and compared. Logistic regression was included as a linear and interpretable baseline model. Its coefficients were later used to build a simple scorecard, which provides a practical way for stakeholders to assess which factors influence a customer's default risk. Tree-based models, including XGBoost and random forest, were used because they can capture nonlinear relationships and interactions among predictors. Neural networks were also included because of their flexibility and ability to model complex patterns. However, because tree-based models and neural networks don't have a closed-form formula compared to logistic regression, the scorecard built for them is a simple manual-based binning.

The dataset was split into training, validation, and test sets. For model selection, each model was trained on the training set, tuned and evaluated on the validation set, and then a final assessment on the test set after final model selection. Since the dataset was imbalanced, accuracy alone was not appropriate to evaluate performance. Therefore, metrics such as ROC-AUC, precision, recall, and F1-score were used. These metrics allowed us to compare overall predictive performance while also evaluating each model's ability to identify default-risk loans.

Precision is the proportion of all the model's positive classifications that are actually positive. In our case, precision represents the proportion of loans classified as default that were actually in default. It can be mathematically defined as :

$$\text{Precision} = \frac{\text{Correctly classified true positive}}{\text{true and false positives}} = \frac{TP}{TP + FP} \quad (3)$$

Recall, also known as the true positive rate(TPR), is the proportion of all actual positives that were classified correctly as positives. For our case, recall represents the proportion of actual default-risk loans classified as default. Mathematically, it can be defined as:

$$\text{Recall} = \frac{\text{Correctly classified true positive}}{\text{all true positives}} = \frac{TP}{TP + FN} \quad (4)$$

The receiver-operating characteristic curve(ROC) is a visual representation of a model's performance across all thresholds.

The ROC curve is drawn by calculating the true positive rate and false positive rate (FPR) at every threshold possible (within the interval $[0,1]$). A perfect model and its threshold can be represented at the point $(0,1)$ of the curve. This means we accurately predict the class of interest with 100% certainty. Area under the curve (AUC) represents the probability that the model assigns a higher predicted probability to a randomly chosen true positive observation than to a randomly chosen true negative observation. AUC and ROC were used not only to evaluate the models, but also for model selection. Due to the heavy imbalance in the dependent variable, AUC alone may provide an overly optimistic view of model performance. Therefore, model selection was conducted in conjunction with precision and recall, as stated previously. Examples of ROC and AUC can be found in 6, 8, 7.

III. RESULTS

Before modeling, some form of exploratory data analysis was needed. For this dataset, I decided to move the EDA post-scorecard build because it contains a large number of features, making it time-consuming to examine each independent variable individually. Instead, the pipeline used feature selection methods to reduce the feature space. From the reduced feature space, we can train the models to assess which variables were more informative and perform EDA around the variables to extract more insights.

To still understand the overall structure of the data, PCA and t-SNE were used as exploratory tools. These methods help visualize whether the observations show linear or nonlinear segmentation between the classes. If the classes are not clearly separated linearly, this helps justify the use of nonlinear models and probabilistic models like logistic regression. PCA and t-SNE also helped with choosing the preprocessing methods. For example, 3 shows the t-SNE plot using only the numerical variables after applying the StandardScaler. The separation looks decent, but since we do not know if all numerical columns are normally distributed, I also used the RobustScaler to reduce the influence of outliers and skewed values. With the RobustScaler, 4 shows more noticeable clustered areas, suggesting that the scaling choice can affect how the data structure appears.

When the categorical variables were added through the full preprocessing pipeline, like in 5, the t-SNE plot became harder to interpret because of the high-cardinality categorical features and one-hot encoding. Because of this, t-SNE was mainly used as an exploratory tool on the numerical preprocessing steps to get a general idea of possible data patterns before modeling.

Data was split into training, validation, and testing using a 70/15/15 split. The trained models were evaluated using metrics focused on class 0, which represents default risk. Data cleaning and feature engineering were performed inside the pipeline to reduce data leakage and ensure that preprocessing steps were learned only from the training data. After evaluating the model results on the validation set, the threshold was tuned to maintain a high class 0 F1-score, while prioritizing high recall because detecting class 0 loans is especially important.

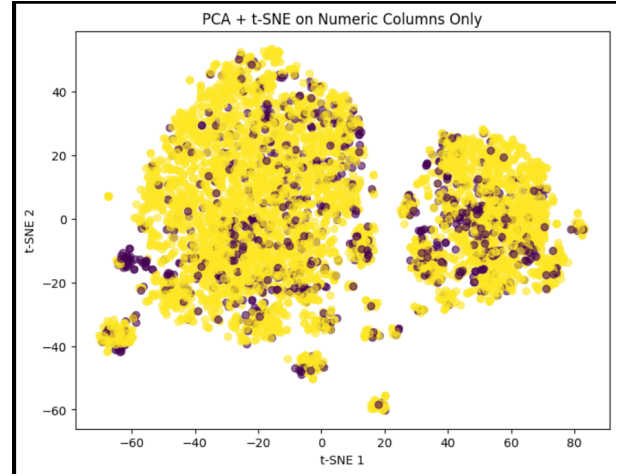


Fig. 3. t-SNE using standard scaler

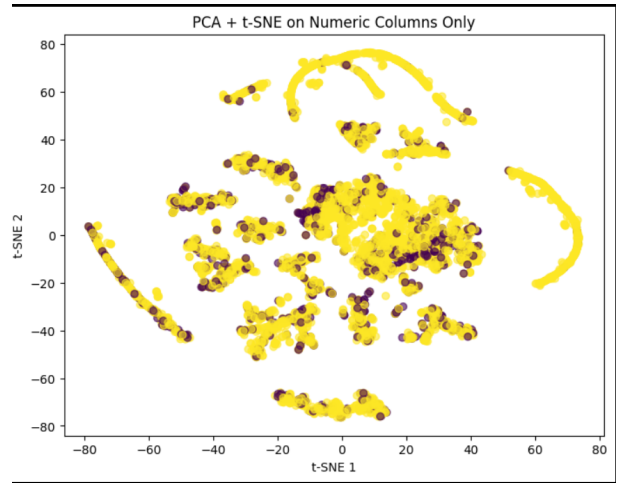


Fig. 4. t-SNE using robust scaler

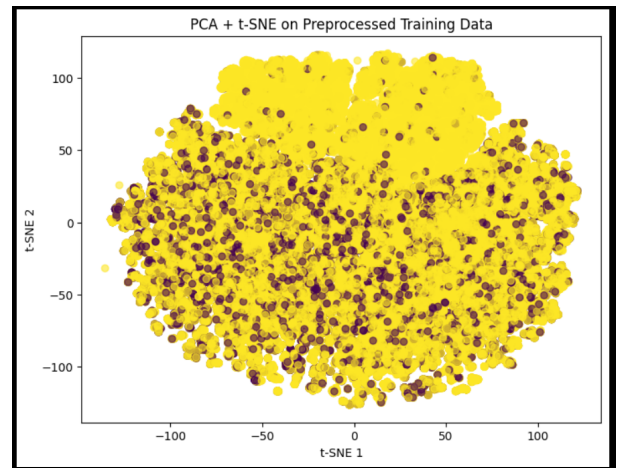


Fig. 5. t-SNE using both numerical and categorical columns

This is because default-risk loans can cause the lender to lose money, so it is important to identify customers who may be at risk. However, some precision must also be maintained to avoid producing too many false alarms.

To address class imbalance, several methods were used for each model. For logistic regression, L2 regularization was used to shrink coefficients toward 0, while class weights, different solvers, were also tested. A similar approach was used for the tree-based models, where different hyperparameters and imbalance handling methods were compared for the respective models. Neural networks were slightly different, since the initial learning rate, weight decay, and architecture size were tuned. Instead of assuming that a larger neural network would automatically perform better, different architecture sizes were tested and compared.

Randomized search was used to keep hyperparameter tuning computationally manageable while maintaining consistency across all models. Specifically, each model was tuned using 10 iterations over a predefined parameter grid and evaluated with 3-fold cross-validation. The results of the best-performing models evaluated by validation are shown in Table I after threshold tuning.

TABLE I
MODEL PERFORMANCE COMPARISON WITH RESPECT TO CLASS 0 IN VALIDATION

Model	ROC-AUC	Precision	Recall	F1-score
Logistic Regression	0.75	0.29	0.60	0.40
Random Forest	0.745	0.29	0.61	0.39
XGBoost	0.75	0.30	0.58	0.40
Neural Network	0.79	0.43	0.49	0.45

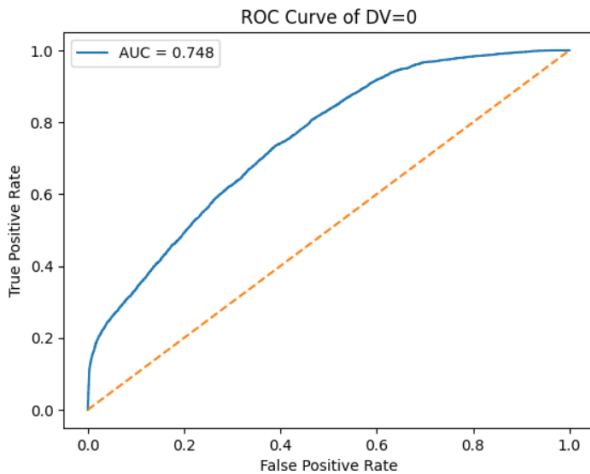


Fig. 6. Logistic regression final model ROC curve on validation set

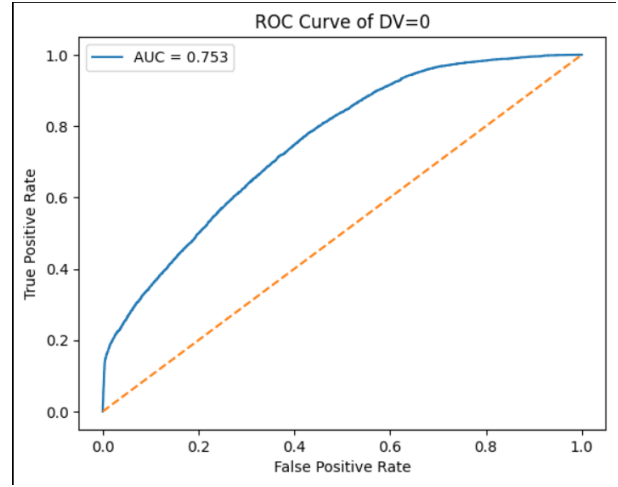


Fig. 7. XGBoost classifier final model ROC curve on validation set

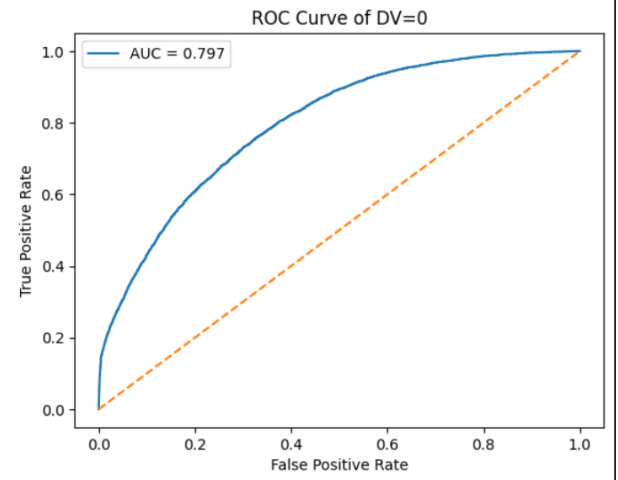


Fig. 8. Neural Network final model ROC curve on validation set

In addition to tuning class-based parameters, sampling methods were also incorporated. More specifically, SMOTE, undersampling, and oversampling were included in the parameter grid. These sampling methods were included only for logistic regression and tree-based models.

The parameter grid for random search had three main versions for logistic regression and the tree-based models. The first version used sampling methods in the pipeline without custom class weights. The second version used custom class weights without any sampling method. The third version combined both sampling methods and custom class weights for completeness. However, this combination is usually not recommended because it can be too aggressive in trying to identify the minority class and may lead to poor generalization.

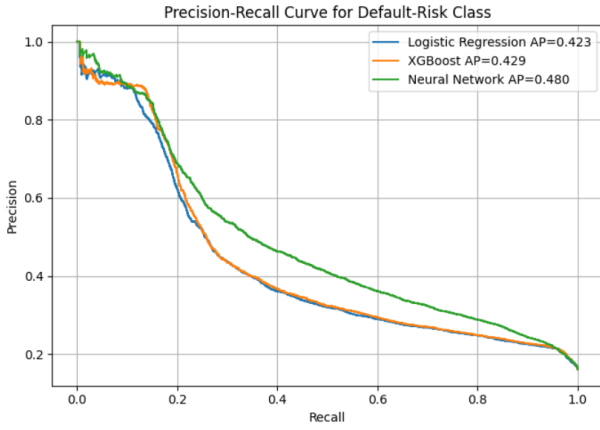


Fig. 9. Final models precision recall curve

A. Final Model Selection and Test Performance

The final models were selected by choosing the best-performing model from each category. This resulted in three final models used for model comparison and scorecard development for stakeholders in Table II. The selected final models were logistic regression with L2 regularization and custom class weights, XGBoost with a sampler, and a large neural network. The validation and test results were similar, suggesting that the selected models generalized reasonably well to unseen data.

TABLE II
FINAL MODEL PERFORMANCE COMPARISON WITH RESPECT TO CLASS 0
IN TEST SET

Model	ROC-AUC	Precision	Recall	F1-score
Logistic Regression	0.75	0.29	0.60	0.39
XGBoost	0.75	0.30	0.59	0.40
Neural Network	0.79	0.38	0.55	0.45

The final stage of the analysis focuses on comparing the best-performing models and creating a scorecard for the logistic regression model using optbinning. The scorecard takes the most important logistic regression variables (capped at 10), bins their values into risk groups, and calculates the event rate for each bin, where the event represents a default loan. Therefore, higher event rates indicate bins with a larger proportion of risky loans.

The scorecard also gives metrics like WoE and IV to help explain how each variable separates good loans from default-risk loans. In simple terms, IV shows how useful a variable is for predicting risk: a higher IV means the variable gives more information to the scorecard, while a lower IV means it is weaker.

TABLE III
TOP SCORECARD VARIABLES BY INFORMATION VALUE

Variable	Highest-Risk Bin	Event Rate	IV
last_fico_range_low	$(-\infty, 572.50)$	0.743	1.518
total_rec_prncp	$(-\infty, 1996.59)$	0.713	0.691
out_prncp	$(-\infty, 14.32)$	0.202	0.066
int_rate	$[17.11, \infty)$	0.304	0.152
verification_status	Verified	0.207	0.028
home_ownership	ANY/RENT	0.187	0.014

Table VI provides an example of how the scorecard interprets one binned variable. For `total_rec_prncp`, loans with values below 1996.59 had an event rate of 0.713, meaning about 71.3% of loans in this group were default-risk loans. Since this bin has a high event rate, it is considered a higher-risk group and receives negative scorecard points. After the scorecard creation, we have evaluated the scorecard using the test set with the following results:

TABLE IV
SCORECARD EVALUATION ON TEST SET

Class	Precision	Recall	F1-score
non-default	0.99	0.93	0.96
Default	0.71	0.93	0.81

The scorecard achieved a high recall of 93% for the default class, meaning that it correctly identified most default-risk loans. However, the precision was lower compared to the non-default class, indicating that some non-default loans were also classified as default-risk. This is an acceptable tradeoff because identifying risky borrowers is more important than minimizing every false alarm.

Along with the scorecard, XGBoost and the neural network were used to estimate PD, or probability of default. A higher PD means the model believes the borrower has a higher chance of being in default or bad standing at the current time. For XGBoost and the neural network, PD was estimated using the model's predicted class probabilities. These probabilities were then grouped into risk bands to make the results easier to interpret. This allows the project to answer the main question using three different modeling approaches: logistic regression/scorecard from statistics, XGBoost from machine learning, and a neural network from deep learning. Overall, these models help compare how well different methods can detect whether a borrower is likely to be in bad standing at the current observation time.

TABLE V
NEURAL NETWORK PROBABILITY OF DEFAULT RISK BANDS

PD Range	Risk Band
$[0.00, 0.10]$	Very Low
$(0.10, 0.25]$	Low
$(0.25, 0.50]$	Medium
$(0.50, 0.75]$	High
$(0.75, 1.00]$	Very High

Note: A similar procedure with the same risk bands was also used for the XGBoost classifier.

IV. DISCUSSION AND CONCLUSION

We aimed to build a model that can detect default-risk loans at the current observation time using three different approaches: a statistical model, a machine learning model, and a deep learning model. More specifically, the goal was to detect whether a borrower is likely to be in default-risk standing at the current time. A practical scorecard was also developed as a current-risk segmentation tool to flag borrowers for closer monitoring and manual review.

From the results, XGBoost and logistic regression showed the most similar and strongest recall performance across both the validation and test sets. The probability of default, or PD, from XGBoost and the neural network was used to assign each borrower a predicted probability of being in default. Since logistic regression is a parametric model, its coefficients were also used to build a scorecard system. This allowed the model to assign scorecard points based on the borrower's characteristics.

Overall, the scorecard gives stakeholders and financial modeling teams a more interpretable way to measure risk. Instead of only predicting whether a borrower may default, the scorecard also shows which features are strong predictors and how different feature ranges increase or decrease default risk.

V. FUTURE WORK

Future work could focus on several improvements. One important improvement would be to obtain more default-risk loan data from other banks or financial institutions. This would create a larger and more diverse pool of default characteristics, which could help the models learn stronger patterns about what factors are most associated with default-risk loans.

Another useful direction would be to further study which variables have the strongest influence on default risk, possibly through additional experiments or more detailed feature analysis. Future work could also focus on loan-origination prediction, meaning predicting default risk before the borrower receives the loan. This would be useful for helping lenders identify risky borrowers earlier and reduce losses before the loan is approved. In future work, validation curves and learning curves could be used to better evaluate hyperparameter tuning and examine how closely each model learns the training data compared with its validation performance. It would also be beneficial to further improve logistic regression by carefully assessing the model assumptions to further support the choice of model and its accuracy.

APPENDIX

TABLE VI
EXAMPLE SCORECARD ROW

Variable	Bin	Event Rate	IV	Points
total_rec_pnrcp	$(-\infty, 1996.59)$	0.713	0.691	-67.65

REFERENCES

- [1] <https://gnpalencia.org/optbinning/scorecard.html>
- [2] A. Karasan, *Machine Learning for Financial Risk Management with Python: Algorithms for Modeling Risk*. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [3] Google Developers, "Classification: ROC and AUC," *Machine Learning Crash Course*. Available: <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>.
- [4] Google Developers, "Classification: Accuracy, Precision, Recall," *Machine Learning Crash Course*. Available: <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>.